

Intro to Data Structures with STL

Sean

May 6, 2026



Outline

- STL and iterator
- pair **and** tuple
- array
- stack **and** queue
- vector **and** deque
- priority_queue
- set **and** multiset
- map
- unordered_set **and** unordered_map



What is a data structure?

- A data structure is a systematic way to store data



What is a data structure?

- A data structure is a systematic way to store data
- Different data structures help with different problems because they change the complexity of the operations you need



What is a data structure?

- A data structure is a systematic way to store data
- Different data structures help with different problems because they change the complexity of the operations you need
- Arrays are already a type of data structure



What is a data structure?

- A data structure is a systematic way to store data
- Different data structures help with different problems because they change the complexity of the operations you need
- Arrays are already a type of data structure
- In competitive programming, harder problems often require stronger data structures to solve them cleanly



STL and iterator

- STL (Standard Template Library)
- A built-in C++ library that contains many different data structures



STL and iterator

- STL (Standard Template Library)
- A built-in C++ library that contains many different data structures
- Works with many different types (int, long long, double, char)
- Different STL components can also be nested together



STL and iterator

- STL (Standard Template Library)
- A built-in C++ library that contains many different data structures
- Works with many different types (int, long long, double, char)
- Different STL components can also be nested together
- It includes containers, iterators, and algorithms
- So let us start with iterators



STL and iterator

- STL (Standard Template Library)
- A built-in C++ library that contains many different data structures
- Works with many different types (int, long long, double, char)
- Different STL components can also be nested together
- It includes containers, iterators, and algorithms
- So let us start with iterators
- For the details, see [cpp reference](#)



iterator

- An iterator is a tool used to access elements inside STL containers
- To get the value pointed to by an iterator, put the dereference operator (*) in front

iterator

- An iterator is a tool used to access elements inside STL containers
- To get the value pointed to by an iterator, put the dereference operator (*) in front
- The iterator categories we will use here are random-access, forward, and bidirectional iterators
 - Random-access iterator: like array indexing with `a[i]`; a random-access iterator can move to any position
 - Forward iterator: it can only move one position forward at a time (`++`)
 - Bidirectional iterator: it can move one position either forward or backward at a time (`--` / `++`)



iterator

- STL has two iterator types we often use: `::iterator` and `::reverse_iterator`
 - With `::iterator`, use `begin()` and `end()` to access the front and the back
 - With `::reverse_iterator`, use `rbegin()` and `rend()` to access the back and the front
 - STL container ranges are left-closed and right-open, so `end()` and `rend()` do not point to actual elements and cannot be dereferenced; use `*--v.end()` if you need the last element



STL

- STL can be roughly divided into four categories
- Sequence containers – array, vector, list, deque
- Container adaptors – stack, queue, priority_queue
- Associative containers – set, map, multiset, multimap
- Unordered associative containers – take the above and add unordered_ in front
- Next, we will go through these STL components one by one



pair

- It is in the header `#include <utility>`

pair

- It is in the header `#include <utility>`
- Declaration: `pair<T1, T2> p`

pair

- It is in the header `#include <utility>`
- Declaration: `pair<T1, T2> p`
- Assignment: `make_pair(T1 a, T2 b)`



pair

- It is in the header `#include <utility>`
- Declaration: `pair<T1, T2> p`
- Assignment: `make_pair(T1 a, T2 b)`
- After C++11, you can also use braces instead
- `pair<T1, T2> p = {a, b}`



pair

- It is in the header `#include <utility>`
- Declaration: `pair<T1, T2> p`
- Assignment: `make_pair(T1 a, T2 b)`
- After C++11, you can also use braces instead
- `pair<T1, T2> p = {a, b}`
- Use `p.first` and `p.second` to get the first and second values



pair

- It is in the header `#include <utility>`
- Declaration: `pair<T1, T2> p`
- Assignment: `make_pair(T1 a, T2 b)`
- After C++11, you can also use braces instead
- `pair<T1, T2> p = {a, b}`
- Use `p.first` and `p.second` to get the first and second values
- `pair` can be compared; by default it compares the first component first, and if they are equal, it compares the second
- You can also define your own `compare` function



pair

- It is in the header `#include <utility>`
- Declaration: `pair<T1, T2> p`
- Assignment: `make_pair(T1 a, T2 b)`
- After C++11, you can also use braces instead
- `pair<T1, T2> p = {a, b}`
- Use `p.first` and `p.second` to get the first and second values
- `pair` can be compared; by default it compares the first component first, and if they are equal, it compares the second
- You can also define your own `compare` function
- Small trick: if you want to sort by the second component, you can swap the first and second components when storing the data



tuple

- What if your data needs three components?

tuple

- What if your data needs three components?
- Nest pairs inside pairs?

tuple

- What if your data needs three components?
- Nest pairs inside pairs?

~~pair<pair<int, int>, int>~~

tuple

- What if your data needs three components?
- Nest pairs inside pairs?

~~pair<pair<int, int>, int>~~

- What if you need even more?

tuple

- What if your data needs three components?
- Nest pairs inside pairs?
~~pair<pair<int, int>, int>~~
- What if you need even more?
- Then you need tuple
- tuple and pair are actually very similar

tuple

- It is in the header `#include <tuple>`



tuple

- It is in the header `#include <tuple>`
- Declaration: `tuple<T1, T2, ..., Tn> t`

tuple

- It is in the header `#include <tuple>`
- Declaration: `tuple<T1, T2, ..., Tn> t`
- Assignment: just replace `make_pair` with `make_tuple`



tuple

- It is in the header `#include <tuple>`
- Declaration: `tuple<T1, T2, ..., Tn> t`
- Assignment: just replace `make_pair` with `make_tuple`
- You can use `get<i>(t)` to get the value of `t` at index `i`



tuple

- It is in the header `#include <tuple>`
- Declaration: `tuple<T1, T2, ..., Tn> t`
- Assignment: just replace `make_pair` with `make_tuple`
- You can use `get<i>(t)` to get the value of `t` at index `i`
- If you think that is too annoying, there are two small tricks



tuple

- It is in the header `#include <tuple>`
- Declaration: `tuple<T1, T2, ..., Tn> t`
- Assignment: just replace `make_pair` with `make_tuple`
- You can use `get<i>(t)` to get the value of `t` at index `i`
- If you think that is too annoying, there are two small tricks
 - Use `tie(a, b, c, ...) = t` so that `a, b, c, ...` correspond to the values in `t`
 - After C++17, you can use `auto [a, b, c, ...] = t` to achieve the same effect



tuple

- It is in the header `#include <tuple>`
- Declaration: `tuple<T1, T2, ..., Tn> t`
- Assignment: just replace `make_pair` with `make_tuple`
- You can use `get<i>(t)` to get the value of `t` at index `i`
- If you think that is too annoying, there are two small tricks
 - Use `tie(a, b, c, ...) = t` so that `a, b, c, ...` correspond to the values in `t`
 - After C++17, you can use `auto [a, b, c, ...] = t` to achieve the same effect
- Sorting works the same way as for `pair`



array

- It is in the header `#include <array>`



array

- It is in the header `#include <array>`
- Declaration: `array<T, size> a`



array

- It is in the header `#include <array>`
- Declaration: `array<T, size> a`
- Assignment: `a = {1, 42, 48763}`



array

- It is in the header `#include <array>`
- Declaration: `array<T, size> a`
- Assignment: `a = {1, 42, 48763}`
- There are two ways to access elements



array

- It is in the header `#include <array>`
- Declaration: `array<T, size> a`
- Assignment: `a = {1, 42, 48763}`
- There are two ways to access elements
 - `a[1] = 42;`
 - `a.at(1) = 42;`



array

- It is in the header `#include <array>`
- Declaration: `array<T, size> a`
- Assignment: `a = {1, 42, 48763}`
- There are two ways to access elements
 - `a[1] = 42;`
 - `a.at(1) = 42;`
- `at()` performs bounds checking, and when you go out of range it will throw an exception



array

- It is in the header `#include <array>`
- Declaration: `array<T, size> a`
- Assignment: `a = {1, 42, 48763}`
- There are two ways to access elements
 - `a[1] = 42;`
 - `a.at(1) = 42;`
- `at()` performs bounds checking, and when you go out of range it will throw an exception
- `[]` is faster than `at()` for access



array

- A C-Style array, that is, something like `int a[5]`
- can decay into a pointer, so when you pass it to a function you may lose the size information



array

- A C-Style array, that is, something like `int a[5]`
- can decay into a pointer, so when you pass it to a function you may lose the size information
- But an STL array can use `.size()` to get its size



array

- A C-Style array, that is, something like `int a[5]`
- can decay into a pointer, so when you pass it to a function you may lose the size information
- But an STL array can use `.size()` to get its size
- Another big advantage is that it can use `for-each` loops
- `for (auto &it : a) std::cout << it;`



vector

- A dynamic array: an array whose length can grow and shrink freely
- It is one of the most frequently used STL containers



vector

- A dynamic array: an array whose length can grow and shrink freely
- It is one of the most frequently used STL containers
- It is in the header `#include <vector>`



vector

- A dynamic array: an array whose length can grow and shrink freely
- It is one of the most frequently used STL containers
- It is in the header `#include <vector>`
- Declaration: `vector<T> v`



vector

- A dynamic array: an array whose length can grow and shrink freely
- It is one of the most frequently used STL containers
- It is in the header `#include <vector>`
- Declaration: `vector<T> v`
- `v.size()`: returns the length of `v`, complexity $\mathcal{O}(1)$
- `v.empty()` returns a `bool` telling whether `v` is empty, complexity $\mathcal{O}(1)$



vector

- A dynamic array: an array whose length can grow and shrink freely
- It is one of the most frequently used STL containers
- It is in the header `#include <vector>`
- Declaration: `vector<T> v`
- `v.size()`: returns the length of `v`, complexity $\mathcal{O}(1)$
- `v.empty()` returns a `bool` telling whether `v` is empty, complexity $\mathcal{O}(1)$
- `v.push_back(T a)` adds `a` to the back of `v`, amortized complexity $\mathcal{O}(1)$
- `v.pop_back()` removes the element at the back of `v`, complexity $\mathcal{O}(1)$



vector

- `v.clear()` clears `v`, complexity $\mathcal{O}(\text{size})$

vector

- `v.clear()` clears `v`, complexity $\mathcal{O}(\text{size})$
- `v.resize(size_type n)` changes the length of `v` to `n`

vector

- `v.clear()` clears `v`, complexity $\mathcal{O}(\text{size})$
- `v.resize(size_type n)` changes the length of `v` to `n`
- `v.assign(size_type n, T a)` turns `v` into an array of length `n` where every element is `a`



vector

- `v.clear()` clears `v`, complexity $\mathcal{O}(\text{size})$
- `v.resize(size_type n)` changes the length of `v` to `n`
- `v.assign(size_type n, T a)` turns `v` into an array of length `n` where every element is `a`
- When `vector` runs out of capacity, its implementation usually doubles the size of the storage, so sometimes `vector` can be slower than a plain array



vector

- `v.clear()` clears `v`, complexity $\mathcal{O}(\text{size})$
- `v.resize(size_type n)` changes the length of `v` to `n`
- `v.assign(size_type n, T a)` turns `v` into an array of length `n` where every element is `a`
- When `vector` runs out of capacity, its implementation usually doubles the size of the storage, so sometimes `vector` can be slower than a plain array
- `string` is actually a lot like `vector<char>`, so `string` supports many operations that `vector` has



vector

- `v.clear()` clears `v`, complexity $\mathcal{O}(\text{size})$
- `v.resize(size_type n)` changes the length of `v` to `n`
- `v.assign(size_type n, T a)` turns `v` into an array of length `n` where every element is `a`
- When `vector` runs out of capacity, its implementation usually doubles the size of the storage, so sometimes `vector` can be slower than a plain array
- `string` is actually a lot like `vector<char>`, so `string` supports many operations that `vector` has
- But it also has some extra optimizations, for example:
 - `s += t` concatenates two strings
 - `getline(cin, s)`



deque

- It is in the header `#include <deque>`

deque

- It is in the header `#include <deque>`
- Think of it as a `vector` that can also push and pop at the front

deque

- It is in the header `#include <deque>`
- Think of it as a `vector` that can also push and pop at the front
- `dq.push_front()` and `dq.pop_front()` are for adding and removing elements at the front



deque

- It is in the header `#include <deque>`
- Think of it as a `vector` that can also push and pop at the front
- `dq.push_front()` and `dq.pop_front()` are for adding and removing elements at the front
- It supports efficient operations at both ends, but usually has worse constants than `vector`



deque

- It is in the header `#include <deque>`
- Think of it as a `vector` that can also push and pop at the front
- `dq.push_front()` and `dq.pop_front()` are for adding and removing elements at the front
- It supports efficient operations at both ends, but usually has worse constants than `vector`
- Use `deque` when you need front operations; otherwise, `vector` is usually the better default



stack and queue

- In general, containers like `std::stack`, `std::queue`, and `std::priority_queue` do not expose iterators
- These belong to container adaptor; they are meant to restrict how you access or modify elements
- So they provide a more limited interface for specific use cases
- instead of the more general iterator-based operations that other containers provide



stack

- Think of this container as a jar: each time, you can only take an item from the top or put an item onto the top
- In other words, 「 Last In, First Out 」 (LIFO)



stack

- Think of this container as a jar: each time, you can only take an item from the top or put an item onto the top
- In other words, 「 Last In, First Out 」 (LIFO)
- It is in the header `#include <stack>`



stack

- Think of this container as a jar: each time, you can only take an item from the top or put an item onto the top
- In other words, 「 Last In, First Out 」 (LIFO)
- It is in the header `#include <stack>`
- Declaration: `stack<T> st`

stack

- Think of this container as a jar: each time, you can only take an item from the top or put an item onto the top
- In other words, 「 Last In, First Out 」 (LIFO)
- It is in the header `#include <stack>`
- Declaration: `stack<T> st`
- `st.size()` and `st.empty()` are the same as `vector`



stack

- Think of this container as a jar: each time, you can only take an item from the top or put an item onto the top
- In other words, 「 Last In, First Out 」 (LIFO)
- It is in the header `#include <stack>`
- Declaration: `stack<T> st`
- `st.size()` and `st.empty()` are the same as `vector`
- `st.top()` returns the most recently pushed element



stack

- Think of this container as a jar: each time, you can only take an item from the top or put an item onto the top
- In other words, 「 Last In, First Out 」 (LIFO)
- It is in the header `#include <stack>`
- Declaration: `stack<T> st`
- `st.size()` and `st.empty()` are the same as `vector`
- `st.top()` returns the most recently pushed element
- `st.push(T a)` pushes the element `a` into `st`



stack

- Think of this container as a jar: each time, you can only take an item from the top or put an item onto the top
- In other words, 「 Last In, First Out 」 (LIFO)
- It is in the header `#include <stack>`
- Declaration: `stack<T> st`
- `st.size()` and `st.empty()` are the same as `vector`
- `st.top()` returns the most recently pushed element
- `st.push(T a)` pushes the element `a` into `st`
- `st.pop()` removes the most recently pushed element



stack

- Think of this container as a jar: each time, you can only take an item from the top or put an item onto the top
- In other words, 「 Last In, First Out 」 (LIFO)
- It is in the header `#include <stack>`
- Declaration: `stack<T> st`
- `st.size()` and `st.empty()` are the same as `vector`
- `st.top()` returns the most recently pushed element
- `st.push(T a)` pushes the element `a` into `st`
- `st.pop()` removes the most recently pushed element
- All of the above operations have complexity $\mathcal{O}(1)$



queue

- Think of this container as a line: each time, people enter at the back of the line, or leave from the front
- In other words, 「 First In, First Out 」 (FIFO)

queue

- Think of this container as a line: each time, people enter at the back of the line, or leave from the front
- In other words, 「 First In, First Out 」 (FIFO)
- It is in the header `#include <queue>`



queue

- Think of this container as a line: each time, people enter at the back of the line, or leave from the front
- In other words, 「 First In, First Out 」 (FIFO)
- It is in the header `#include <queue>`
- Declaration: `queue<T> q`



queue

- Think of this container as a line: each time, people enter at the back of the line, or leave from the front
- In other words, 「 First In, First Out 」 (FIFO)
- It is in the header `#include <queue>`
- Declaration: `queue<T> q`
- `q.size()` and `q.empty()` are the same as `vector`



queue

- Think of this container as a line: each time, people enter at the back of the line, or leave from the front
- In other words, 「 First In, First Out 」 (FIFO)
- It is in the header `#include <queue>`
- Declaration: `queue<T> q`
- `q.size()` and `q.empty()` are the same as `vector`
- `q.front()` returns the first element that was pushed
- `q.back()` returns the most recently pushed element



queue

- Think of this container as a line: each time, people enter at the back of the line, or leave from the front
- In other words, 「 First In, First Out 」 (FIFO)
- It is in the header `#include <queue>`
- Declaration: `queue<T> q`
- `q.size()` and `q.empty()` are the same as `vector`
- `q.front()` returns the first element that was pushed
- `q.back()` returns the most recently pushed element
- `q.push(T a)` pushes the element `a` into `q`



queue

- Think of this container as a line: each time, people enter at the back of the line, or leave from the front
- In other words, 「 First In, First Out 」 (FIFO)
- It is in the header `#include <queue>`
- Declaration: `queue<T> q`
- `q.size()` and `q.empty()` are the same as `vector`
- `q.front()` returns the first element that was pushed
- `q.back()` returns the most recently pushed element
- `q.push(T a)` pushes the element `a` into `q`
- `q.pop()` removes the first element that was pushed



queue

- Think of this container as a line: each time, people enter at the back of the line, or leave from the front
- In other words, 「 First In, First Out 」 (FIFO)
- It is in the header `#include <queue>`
- Declaration: `queue<T> q`
- `q.size()` and `q.empty()` are the same as `vector`
- `q.front()` returns the first element that was pushed
- `q.back()` returns the most recently pushed element
- `q.push(T a)` pushes the element `a` into `q`
- `q.pop()` removes the first element that was pushed
- All of the above operations have complexity $\mathcal{O}(1)$



queue

- You can also implement the functionality of a queue using `vector`
- The constant factor is a bit smaller, and you can also keep track of what is inside the queue



queue

- You can also implement the functionality of a queue using vector
- The constant factor is a bit smaller, and you can also keep track of what is inside the queue

```
1  vector<int> q;  
2  int idx = 0;  
3  while (q.size() > idx) {  
4      int x;  
5      cin >> x;  
6      q.push_back(x); // push()  
7      cout << q[idx] << '\n'; // front()  
8      cout << q.back() << '\n'; // back()  
9      idx++; // pop()  
10 }
```



priority_queue

- `priority_queue` is built from a binary heap, and is used to keep track of the current most extreme value
- Binary heap visualization



priority_queue

- `priority_queue` is built from a binary heap, and is used to keep track of the current most extreme value
- Binary heap visualization
- It is in the header `#include <queue>`



priority_queue

- `priority_queue` is built from a binary heap, and is used to keep track of the current most extreme value
- Binary heap visualization
- It is in the header `#include <queue>`
- Declaration: `priority_queue<T, Container, cmp>`.
T is the type, Container is the underlying container, usually vector, and cmp is the comparison rule



priority_queue

- `priority_queue` is built from a binary heap, and is used to keep track of the current most extreme value
- Binary heap visualization
- It is in the header `#include <queue>`
- Declaration: `priority_queue<T, Container, cmp>`.
T is the type, Container is the underlying container, usually vector, and cmp is the comparison rule
 - `priority_queue<int, vector<int>, less<int>>`
pq: maximum value comes out first
 - `priority_queue<int, vector<int>, greater<int>>` pq: minimum value comes out first



priority_queue

- `priority_queue` is built from a binary heap, and is used to keep track of the current most extreme value
- Binary heap visualization
- It is in the header `#include <queue>`
- Declaration: `priority_queue<T, Container, cmp>`.
T is the type, Container is the underlying container, usually vector, and cmp is the comparison rule
 - `priority_queue<int, vector<int>, less<int>>`
pq: maximum value comes out first
 - `priority_queue<int, vector<int>, greater<int>>` pq: minimum value comes out first
- You can also build it from an iterator range, for example `priority_queue<int> pq(first, last)`, which inserts all elements in *[first, last)*



priority_queue

- `pq.size()`, `pq.empty()`, and `pq.top()` are used the same way as in other STL containers
- A simple rule: `less` gives a max-heap, and `greater` gives a min-heap



priority_queue

- `pq.size()`, `pq.empty()`, and `pq.top()` are used the same way as in other STL containers
- A simple rule: `less` gives a max-heap, and `greater` gives a min-heap
- `pq.push(x)` pushes `x` into `pq`, $\mathcal{O}(\log \text{size})$



priority_queue

- `pq.size()`, `pq.empty()`, and `pq.top()` are used the same way as in other STL containers
- A simple rule: `less` gives a max-heap, and `greater` gives a min-heap
- `pq.push(x)` pushes `x` into `pq`, $\mathcal{O}(\log \text{size})$
- `pq.pop()` removes the top element, $\mathcal{O}(\log \text{size})$



priority_queue

- `pq.size()`, `pq.empty()`, and `pq.top()` are used the same way as in other STL containers
- A simple rule: `less` gives a max-heap, and `greater` gives a min-heap
- `pq.push(x)` pushes `x` into `pq`, $\mathcal{O}(\log size)$
- `pq.pop()` removes the top element, $\mathcal{O}(\log size)$
- Building from a range is $\mathcal{O}(size)$, while pushing elements one by one is $\mathcal{O}(size \log size)$



priority_queue example

CSES 1161 Stick Divisions

There is a stick of length n , and it needs to be divided into m sticks with lengths $d_1, d_2, \dots, d_m$.

Each time, you may split one stick into two, and the cost is the length of that stick. Find the minimum possible total cost.

[link](#)

priority_queue example

CSES 1161 Stick Divisions

There is a stick of length n , and it needs to be divided into m sticks with lengths $d_1, d_2, \dots, d_m$.

Each time, you may split one stick into two, and the cost is the length of that stick. Find the minimum possible total cost.

[link](#)

- Think in reverse: instead of splitting, think of merging

priority_queue example

CSES 1161 Stick Divisions

There is a stick of length n , and it needs to be divided into m sticks with lengths $d_1, d_2, \dots, d_m$.

Each time, you may split one stick into two, and the cost is the length of that stick. Find the minimum possible total cost.

[link](#)

- Think in reverse: instead of splitting, think of merging
- You can see that merging the two smallest sticks each time is the optimal strategy

priority_queue example

CSES 1161 Stick Divisions

There is a stick of length n , and it needs to be divided into m sticks with lengths $d_1, d_2, \dots, d_m$.

Each time, you may split one stick into two, and the cost is the length of that stick. Find the minimum possible total cost.

[link](#)

- Think in reverse: instead of splitting, think of merging
- You can see that merging the two smallest sticks each time is the optimal strategy
- Implement it with a `priority_queue` that extracts the minimum value



priority_queue example

CSES 1161 Stick Divisions

There is a stick of length n , and it needs to be divided into m sticks with lengths $d_1, d_2, \dots, d_m$.

Each time, you may split one stick into two, and the cost is the length of that stick. Find the minimum possible total cost.

[link](#)

- Think in reverse: instead of splitting, think of merging
- You can see that merging the two smallest sticks each time is the optimal strategy
- Implement it with a `priority_queue` that extracts the minimum value
- Each time, take `pq.top()` twice to get the two smallest values, add them, and `push` the sum back into `pq`



set

- `set` is built from a balanced binary search tree (Balanced Binary Search Tree), so in simple terms it is a tree-shaped data structure
- Binary Search Tree (BST) visualization

set

- `set` is built from a balanced binary search tree (Balanced Binary Search Tree), so in simple terms it is a tree-shaped data structure
- Binary Search Tree (BST) visualization
- It is similar to the mathematical notion of a set: it does not contain duplicate elements



set

- `set` is built from a balanced binary search tree (Balanced Binary Search Tree), so in simple terms it is a tree-shaped data structure
- Binary Search Tree (BST) visualization
- It is similar to the mathematical notion of a set: it does not contain duplicate elements
- It supports insertion, deletion, and search in $\mathcal{O}(\log \textit{size})$



set

- `set` is built from a balanced binary search tree (Balanced Binary Search Tree), so in simple terms it is a tree-shaped data structure
- Binary Search Tree (BST) visualization
- It is similar to the mathematical notion of a set: it does not contain duplicate elements
- It supports insertion, deletion, and search in $\mathcal{O}(\log \text{size})$
- It is in the header `#include <set>`



set

- `set` is built from a balanced binary search tree (Balanced Binary Search Tree), so in simple terms it is a tree-shaped data structure
- Binary Search Tree (BST) visualization
- It is similar to the mathematical notion of a set: it does not contain duplicate elements
- It supports insertion, deletion, and search in $\mathcal{O}(\log \text{size})$
- It is in the header `#include <set>`
- Declaration: `set<T> s`



set

- `s.size()` and `s.empty()` are the same as for other containers

set

- `s.size()` and `s.empty()` are the same as for other containers
- `s.insert(T val)` inserts `val` into `s`, $\mathcal{O}(\log \text{size})$

set

- `s.size()` and `s.empty()` are the same as for other containers
- `s.insert(T val)` inserts `val` into `s`, $\mathcal{O}(\log \textit{size})$
- `s.find(T val)` finds the iterator pointing to `val`; if it does not exist, it returns `s.end()`, $\mathcal{O}(\log \textit{size})$
- `s.erase(T val)` erases `val` from `s`, $\mathcal{O}(\log \textit{size})$



set

- `s.size()` and `s.empty()` are the same as for other containers
- `s.insert(T val)` inserts `val` into `s`, $\mathcal{O}(\log \textit{size})$
- `s.find(T val)` finds the iterator pointing to `val`; if it does not exist, it returns `s.end()`, $\mathcal{O}(\log \textit{size})$
- `s.erase(T val)` erases `val` from `s`, $\mathcal{O}(\log \textit{size})$
- `s.erase(iterator first, iterator last)` erases the elements in $[first, last)$, $\mathcal{O}(\textit{count})$



set

- `s.size()` and `s.empty()` are the same as for other containers
- `s.insert(T val)` inserts `val` into `s`, $\mathcal{O}(\log \textit{size})$
- `s.find(T val)` finds the iterator pointing to `val`; if it does not exist, it returns `s.end()`, $\mathcal{O}(\log \textit{size})$
- `s.erase(T val)` erases `val` from `s`, $\mathcal{O}(\log \textit{size})$
- `s.erase(iterator first, iterator last)` erases the elements in $[first, last)$, $\mathcal{O}(\textit{count})$
- `s.count(T val)` gets the number of occurrences of `val` in `s`, $\mathcal{O}(\log \textit{size} + \textit{count})$



set

- `s.lower_bound(T val)` finds the first iterator in `s` that points to an element $\geq val$, $\mathcal{O}(\log size)$
- `s.upper_bound(T val)` finds the first iterator in `s` that points to an element $> val$, $\mathcal{O}(\log size)$



set

- `s.lower_bound(T val)` finds the first iterator in `s` that points to an element $\geq val$, $\mathcal{O}(\log size)$
- `s.upper_bound(T val)` finds the first iterator in `s` that points to an element $> val$, $\mathcal{O}(\log size)$
- `set::iterator` is a bidirectional iterator, and the elements are kept in increasing order; moving the iterator forward or backward has complexity $\mathcal{O}(1)$



multiset

- multiset is a set that allows duplicate elements



multiset

- `multiset` is a set that allows duplicate elements
- It is used exactly the same way as `set`

multiset

- `multiset` is a set that allows duplicate elements
- It is used exactly the same way as `set`
- If the argument passed to `s.erase(val)` is a value instead of an iterator, then it will erase every `val` in `s`

multiset

- `multiset` is a set that allows duplicate elements
- It is used exactly the same way as `set`
- If the argument passed to `s.erase(val)` is a value instead of an iterator, then it will erase every `val` in `s`
- So if you want to erase only one element, you need to pass an iterator; in implementations this is often used together with `lower_bound` and `upper_bound`



multiset example

CSES 1076 Sliding Median

Find the median of every subarray of length k . [link](#)

multiset example

CSES 1076 Sliding Median

Find the median of every subarray of length k . [link](#)

- If instead we were asking for the median of the prefix from the first number to the i -th number in a sequence of length n , then because numbers are only being added, we would not need to maintain a window size, meaning there would be no expired elements, so we could use `priority_queue` directly



multiset example

CSES 1076 Sliding Median

Find the median of every subarray of length k . [link](#)

- If instead we were asking for the median of the prefix from the first number to the i -th number in a sequence of length n , then because numbers are only being added, we would not need to maintain a window size, meaning there would be no expired elements, so we could use `priority_queue` directly
- But now we need to handle expired elements, and we need to delete a specified value, so we need `multiset`, which supports `find`



multiset example

CSES 1076 Sliding Median

Find the median of every subarray of length k . [link](#)

- If instead we were asking for the median of the prefix from the first number to the i -th number in a sequence of length n , then because numbers are only being added, we would not need to maintain a window size, meaning there would be no expired elements, so we could use `priority_queue` directly
- But now we need to handle expired elements, and we need to delete a specified value, so we need `multiset`, which supports `find`
- The implementation is very similar to `priority_queue`; it only adds the operation of deleting a specified element



map

- It consists of key and value; each key corresponds to exactly one value



map

- It consists of `key` and `value`; each `key` corresponds to exactly one `value`
- Like `set`, it is a tree structure and supports insertion, deletion, and search in $\mathcal{O}(\log \textit{size})$



map

- It consists of `key` and `value`; each `key` corresponds to exactly one `value`
- Like `set`, it is a tree structure and supports insertion, deletion, and search in $\mathcal{O}(\log size)$
- It is in the header `#include <map>`



map

- It consists of **key** and **value**; each key corresponds to exactly one value
- Like `set`, it is a tree structure and supports insertion, deletion, and search in $\mathcal{O}(\log \textit{size})$
- It is in the header `#include <map>`
- `map<T1, T2> mp` declares a map whose key type is `T1` and whose value type is `T2`; you can think of it as `set<pair<T1, T2>>`



map

- It consists of **key** and **value**; each key corresponds to exactly one value
- Like `set`, it is a tree structure and supports insertion, deletion, and search in $\mathcal{O}(\log \textit{size})$
- It is in the header `#include <map>`
- `map<T1, T2> mp` declares a map whose key type is `T1` and whose value type is `T2`; you can think of it as `set<pair<T1, T2>>`
- `map` is actually a structure made of `pair`, so the element pointed to by an iterator is a `pair`; to get the key, use `first`, and to get the value, use `second`



map

- It consists of key and value; each key corresponds to exactly one value
- Like set, it is a tree structure and supports insertion, deletion, and search in $\mathcal{O}(\log \text{size})$
- It is in the header `#include <map>`
- `map<T1, T2> mp` declares a map whose key type is T1 and whose value type is T2; you can think of it as `set<pair<T1, T2>>`
- map is actually a structure made of pair, so the element pointed to by an iterator is a pair; to get the key, use `first`, and to get the value, use `second`
- In competitive programming, map is useful when the keys are not small consecutive integers, so direct indexing with an array or vector is inconvenient



map

- Its general usage is the same as `set`

map

- Its general usage is the same as set
- `mp.insert(pair<T1 a, T2 b>)` inserts an element into mp; a is the key, and b is the value



map

- Its general usage is the same as set
- `mp.insert(pair<T1 a, T2 b>)` inserts an element into mp; a is the key, and b is the value
- `mp[T1 key]` queries what value corresponds to key, $\mathcal{O}(\log size)$
- `mp[T1 key] = value` modifies the value corresponding to key, $\mathcal{O}(\log size)$



map

- Its general usage is the same as set
- `mp.insert(pair<T1 a, T2 b>)` inserts an element into mp; a is the key, and b is the value
- `mp[T1 key]` queries what value corresponds to key, $\mathcal{O}(\log size)$
- `mp[T1 key] = value` modifies the value corresponding to key, $\mathcal{O}(\log size)$
 - Note that if key is not originally in mp, it will automatically insert (key, 0)



map

- Its general usage is the same as `set`
- `mp.insert(pair<T1 a, T2 b>)` inserts an element into `mp`; `a` is the key, and `b` is the value
- `mp[T1 key]` queries what value corresponds to `key`, $\mathcal{O}(\log size)$
- `mp[T1 key] = value` modifies the value corresponding to `key`, $\mathcal{O}(\log size)$
 - Note that if `key` is not originally in `mp`, it will automatically insert `(key, 0)`
- Typical uses: frequency counting on large or sparse values, string keys, and cases where direct indexing is inconvenient



map

- `multimap` is a less common version of `map` that allows multiple values for the same key

map

- `multimap` is a less common version of `map` that allows multiple values for the same key
- Useful when duplicate keys must stay sorted, such as events by time, intervals by endpoint, or multiple records under one key

unordered_set, unordered_map

- They use `hash`, so lookup, insertion, and deletion are average $\mathcal{O}(1)$

unordered_set, unordered_map

- They use `hash`, so lookup, insertion, and deletion are average $\mathcal{O}(1)$
- The elements are not ordered, so you cannot use `lower_bound`, `upper_bound`, or `ordered` traversal
- Their iterators are forward iterators



unordered_set, unordered_map

- They use `hash`, so lookup, insertion, and deletion are average $\mathcal{O}(1)$
- The elements are not ordered, so you cannot use `lower_bound`, `upper_bound`, or `ordered` traversal
- Their iterators are forward iterators
- `hash` has collision risk, so worst-case performance can become much worse



unordered_set, unordered_map

- They use `hash`, so lookup, insertion, and deletion are average $\mathcal{O}(1)$
- The elements are not ordered, so you cannot use `lower_bound`, `upper_bound`, or `ordered` traversal
- Their iterators are forward iterators
- `hash` has collision risk, so worst-case performance can become much worse
- In competitive programming, use `unordered_*` when you want fast average lookup; use `set` or `map` when you need order or more predictable behavior



unordered_set, unordered_map

- They use `hash`, so lookup, insertion, and deletion are average $\mathcal{O}(1)$
- The elements are not ordered, so you cannot use `lower_bound`, `upper_bound`, or `ordered` traversal
- Their iterators are forward iterators
- `hash` has collision risk, so worst-case performance can become much worse
- In competitive programming, use `unordered_*` when you want fast average lookup; use `set` or `map` when you need order or more predictable behavior
- In contests like `codeforces` where post-contest hacks exist, a custom `hash` function is often important



custom hash

```
1 struct custom_hash {  
2     static uint64_t splitmix64(uint64_t x) {  
3         x += 0x9e3779b97f4a7c15;  
4         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;  
5         x = (x ^ (x >> 27)) * 0x94d049bb133111eb;  
6         return x ^ (x >> 31);  
7     }  
8  
9     size_t operator()(uint64_t x) const {  
10         static const uint64_t FIXED_RANDOM =  
11             ↪ chrono::steady_clock::now().time_since_epoch().count()  
12         return splitmix64(x + FIXED_RANDOM);  
13     }  
};
```



Choosing a container

- vector: dynamic array
- stack: LIFO
- queue: FIFO
- deque: operations at both ends
- priority_queue: current max/min
- set, multiset: ordered values
- map: ordered key-value
- unordered_*: hash-based, average $\mathcal{O}(1)$



The End

Thanks for listening ~

© 2026 Sean. Please do not repost or redistribute without permission.

